



Build It Yourself: The Imam Al-Shafi'i Academy App

A step-by-step Flutter + Firebase learning guide

Write every screen and service yourself, in the right order, with the reasoning and animation techniques behind each one.

How to use this guide

This guide walks through building the academy app from an empty Flutter project to a working, Firebase-backed app with three roles. Each step tells you exactly what to build, why it comes at that point in the sequence, which Flutter/Firebase concept it teaches, and includes one small, focused code example — not the full file, just the pattern to apply.

Work through the steps in order. Each one depends on something built in the previous step (the Dashboard needs the data model; the data model needs Firebase connected; Firebase needs the project scaffolded). Skipping ahead usually means back-filling something you skipped.

Recommended pace

- Steps 0–2 (setup, splash, login UI): one sitting — no Firebase logic yet, just layout and animation.
- Steps 3–5 (Firebase connection, data model, routing): the "plumbing" — less visual, but everything else depends on it.
- Steps 6–10 (student app): the bulk of the UI work.
- Steps 11–12 (mentor + admin): mostly the same patterns as step 6–10, reapplied.
- Steps 13–16 (Cloud Functions, reminders, security rules, deployment): backend and hardening.

Prerequisites

- Flutter SDK installed and `flutter doctor` passing.
- A Firebase project created (Console → Add project) with Blaze plan enabled for later steps.
- Basic familiarity with Dart syntax — this guide teaches Flutter/Firebase patterns, not Dart itself.

STEP 0

Project & Firebase Setup

Goal: Get an empty Flutter project running, with the folder structure ready, before writing any screen.

What you'll do

- Create the Flutter project and set up the feature-based folder structure (app/, core/, models/, auth/, student/, mentor/, admin/, shared/).
- Add the theme colors and fonts now, even though no screen uses them yet — it saves you from re-styling every screen later.
- Leave Firebase connection for Step 3 — don't install Firebase packages yet. Get something on screen first.

```
flutter create imam_shafii_academy --org com.imamshafii
cd imam_shafii_academy

mkdir -p lib/app lib/core/constants lib/core/services lib/core/widgets
mkdir -p lib/models lib/auth/login lib/auth/splash
mkdir -p lib/student lib/mentor lib/admin lib/shared/widgets lib/shared/layouts
```

Define your brand colors first

Create lib/core/constants/app_colors.dart with your palette as static const Color values. Every screen you build from here on references these constants — never a hard-coded hex value inside a screen file.

```
class AppColors {
  AppColors._();
  static const navy = Color(0xFF1B2559);
  static const gold = Color(0xFFC9A227);
  static const bg   = Color(0xFFF7F8FA);
}
```

 **Why constants first:** Once three or four screens exist, changing a color means one line here instead of a find-and-replace across the project.

 **Try it yourself:** Add a `google\_fonts` dependency and set up a basic ThemeData in lib/app/theme.dart using AppColors, even before any screen consumes it.

STEP 1

The Splash Screen

Goal: Build the very first screen the user sees — pure UI, no Firebase yet — and introduce your first two animations.

Why start here

The splash screen has zero data dependencies, so it's the safest place to practice layout and animation before anything needs to talk to Firebase. It also establishes the visual identity (logo, gradient, motto) that the login screen will echo.

What to build

- A full-screen gradient container (navy → darker navy).
- The academy logo centered, on a white rounded card so it reads clearly against the dark background.
- App name + motto text beneath it.
- A loading spinner at the bottom (this is where you'll later check auth state and navigate away).

 **Animation for this screen:** Wrap the logo container in a Hero widget with a shared tag (e.g. "app-logo"). When you build the Login screen in Step 2, give its logo the same Hero tag — Flutter will automatically animate the logo flying and resizing between the two screens the first time you navigate, at no extra code cost.

```
Hero(  
  tag: 'app-logo',  
  child: Container(  
    width: 92, height: 92,  
    decoration: BoxDecoration(color: Colors.white, borderRadius:  
BorderRadius.circular(24)),  
    child: Image.asset('assets/logo/logo.jpg'),  
  ),  
)
```

Add a second, subtler animation: fade the motto text in after the logo, using `AnimatedOpacity` or a simple `Future.delayed` + `setState` toggling a bool.

```
AnimatedOpacity(  
  opacity: _mottoVisible ? 1 : 0,  
  duration: const Duration(milliseconds: 600),  
  child: Text('لا يُستطاعُ العلمُ براحَةِ الجسد'),  
)
```

 **Try it yourself:** Make the loading spinner switch to a checkmark icon with an `AnimatedSwitcher` once a 2-second `Future.delayed` completes, before you have any real auth check to wait for.

STEP 2

The Login Screen (UI first)

Goal: Build the login form and its visual polish before wiring it to Firebase Auth — keep UI and logic separable.

What to build

- The same logo mark, using the matching Hero tag from Step 1, so navigating from splash to login animates it into place.
- Email + password TextFields using your theme's InputDecorationTheme.
- A submit button that shows a loading spinner in place of its label while "signing in".
- An inline error message area.

 **Hero animation here:** Because the Splash screen's logo Container is wrapped in Hero(tag: 'app-logo', ...), wrapping the Login screen's logo in Hero(tag: 'app-logo', ...) too is all that's needed — no extra animation controller code. Flutter matches widgets by tag across the two routes automatically during navigation.

Give the button real feedback

Before Firebase exists, fake the loading state with a local bool and a 1-second Future.delayed, so you can already see and tune the loading animation.

```
ElevatedButton(  
  onPressed: _loading ? null : _submit,  
  child: _loading  
    ? const SizedBox(  
      width: 18, height: 18,  
      child: CircularProgressIndicator(strokeWidth: 2, color: Colors.white),  
    )  
    : const Text('دخول'),  
)
```

 **Error message animation:** Use AnimatedSize or AnimatedContainer around the error Text so it grows/shrinks smoothly into view instead of the layout jumping when the error appears.

 **Try it yourself:** Add a subtle "shake" animation on the password field when login fails — an AnimationController driving a horizontal Transform.translate offset that oscillates and decays is the classic pattern.

STEP 3

Connect Firebase Authentication

Goal: Wire the login button you already built to a real Firebase project.

What to do

- Run `flutterfire configure` — this generates lib/firebase_options.dart for you. Never hand-write this file.
- Call `Firebase.initializeApp()` in `main()` before `runApp()`.
- Write a small `AuthService` class wrapping `FirebaseAuth.instance` — never call `FirebaseAuth.instance` directly from a screen.

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  runApp(const MyApp());
}

class AuthService {
  final _auth = FirebaseAuth.instance;
  Future<UserCredential> signIn(String email, String password) {
    return _auth.signInWithEmailAndPassword(email: email, password: password);
  }
  Stream<User?> get authStateChanges => _auth.authStateChanges();
}
```

 **Why a service class:** Every screen that needs auth calls `AuthService`, never `FirebaseAuth` directly. Later, when you add Riverpod, you wrap this one class in one provider — and every screen that already depends on the service keeps working unchanged.

 **Try it yourself:** Replace the fake `Future.delayed` loading state from Step 2 with a real call to `authService.signIn(...)`, catching `FirebaseAuthException` to show a real error message.

STEP 4

Design the Firestore Data Model

Goal: Decide your collections and write the Dart model classes before writing any screen that reads or writes data.

Why model classes before screens

If you start querying Firestore directly from a StreamBuilder with raw Map<String, dynamic> data, every screen ends up repeating the same field-name strings — one typo and a screen silently shows nothing. A model class with fromMap/toMap catches that at compile time instead.

The core collections for this app

- users/{uid} — role: student | mentor | admin
- students/{uid} — profile fields + academicYearId + mentorId
- academicYears/year_1, year_2, year_3 — learningDay1, learningDay2, lessonStartTime
- lessons, books, videos, updates — each carries academicYearId + isPublished
- events, notifications — each carries targetAudience

```
class StudentModel {
  final String uid, firstName, lastName, academicYearId, mentorId;
  StudentModel({required this.uid, required this.firstName,
    required this.lastName, required this.academicYearId,
    required this.mentorId});

  factory StudentModel.fromMap(String uid, Map<String, dynamic> map) {
    return StudentModel(
      uid: uid,
      firstName: map['firstName'] ?? '',
      lastName: map['lastName'] ?? '',
      academicYearId: map['academicYearId'] ?? 'year_1',
      mentorId: map['mentorId'] ?? '',
    );
  }

  Map<String, dynamic> toMap() => {
    'firstName': firstName, 'lastName': lastName,
    'academicYearId': academicYearId, 'mentorId': mentorId,
  };
}
```

 **Naming discipline:** Pick exact field names now (academicYearId, not yearId or academic_year) and use them identically in every model, every Firestore document, and every security rule. Mismatches here are the single most common source of "why is my StreamBuilder empty" bugs.

 **Try it yourself:** Write LessonModel, BookModel, and VideoModel the same way — notice how similar the pattern is each time.

STEP 5

Providers & Role-Based Routing

Goal: Introduce Riverpod for state, and `go_router` for navigation that redirects each role to its own home screen.

The chain you're building

Auth state (signed in / out) → the signed-in user's role document → which "home" route to redirect to. Each link in that chain is one Riverpod provider watching the one before it.

```
final authStateProvider = StreamProvider<User?>((ref) {
  return ref.watch(authServiceProvider).authStateChanges;
});

final currentUserDocProvider = StreamProvider<UserModel?>((ref) {
  final auth = ref.watch(authStateProvider).value;
  if (auth == null) return Stream.value(null);
  return ref.watch(authServiceProvider).watchCurrentUserDoc();
});
```

go_router redirect

GoRouter's redirect callback runs on every navigation attempt. Read the two providers above (synchronously, via `ref.read`) and send the user to `/student`, `/mentor`, or `/admin` based on role — or to `/login` if signed out.

```
redirect: (context, state) {
  final user = ref.read(authStateProvider).value;
  if (user == null) return '/login';
  final role = ref.read(currentUserDocProvider).value?.role;
  if (role == 'mentor') return '/mentor';
  if (role == 'admin') return '/admin';
  return '/student';
}
```

 **refreshListenable:** redirect only re-runs on navigation, not automatically when a stream emits. Pass a `ChangeNotifier` as `refreshListenable` that calls `notifyListeners()` inside `ref.listen(...)` on both providers — that's what makes login instantly redirect without the user tapping anything else.

 **Try it yourself:** Sign in with a test account that has no `users/{uid}` document yet and confirm the redirect sends it somewhere sensible instead of crashing.

STEP 6

The Student Shell (bottom navigation)

Goal: Build the 4-tab shell that wraps every student screen — before building what's inside each tab.

Why the shell first

Building the shell before its content means you can navigate between four placeholder screens and get the tab-switching feel right — the active-tab color, the icon animation — before worrying about data on any of them.

```
Scaffold(
  body: child, // the current tab's screen, provided by go_router's ShellRoute
  bottomNavigationBar: BottomNavigationBar(
    currentIndex: _indexForLocation(location),
    onTap: (i) => context.go(_tabs[i]),
    selectedItemColor: AppColors.gold,
    items: const [
      BottomNavigationBarItem(icon: Icon(Icons.home_rounded), label: 'الرئيسية'),
      // ... 3 more
    ],
  ),
)
```

 **Implicit animation for free:** BottomNavigationBar already animates the selected icon/label color change and (with type: .shifting) a background color sweep — you get motion here without writing an AnimationController.

 **Try it yourself:** Temporarily make each of the 4 tabs just a Center(child: Text('tab name')) screen, and confirm switching tabs feels instant with no flicker before building real content.

STEP 7

The Student Dashboard

Goal: The first screen with real data — combine multiple Firestore streams filtered by the student's own `academicYearId`.

The core pattern: read the student's own year first

Every content stream on this screen needs the signed-in student's `academicYearId`. Watch that once at the top of the widget, then pass it into every `StreamBuilder` below — never re-derive it per-section.

```
final student = ref.watch(currentStudentProvider).value;
if (student == null) return const CircularProgressIndicator();

StreamBuilder(
  stream: firestoreService.watchLessonsForStudent(student.academicYearId),
  builder: (context, snapshot) { /* ... */ },
)
```

 **Animate the random quote:** Wrap the motivational quote `Text` in an `AnimatedSwitcher` keyed by the quote text. Even though the quote is picked once per screen build here, this pattern is what you'd use if you later let the user tap to cycle quotes — the old quote fades out while the new one fades in.

```
AnimatedSwitcher(
  duration: const Duration(milliseconds: 400),
  child: Text(quote.text, key: ValueKey(quote.text)),
)
```

 **Staggered section entrance:** Wrap each dashboard section (quote, notifications, lessons, books...) in a small reusable `FadeSlideIn` widget that plays a fade+slide-up animation with an increasing delay per section index — this is what makes a dashboard feel considered rather than static.

 **Try it yourself:** Build that `FadeSlideIn` widget once as a shared widget in `lib/shared/widgets/` — you'll reuse it on almost every list screen from here on.

STEP 8

Content Screens: Lessons, Books, Videos, Updates

Goal: Four very similar list screens — build one carefully, then repeat the pattern.

The shared shape

Each screen is: read the student's academicYearId → StreamBuilder on the matching Firestore query (already filtered to isPublished == true and that year) → ListView.separated → EmptyState widget when the list is empty.

```
StreamBuilder<List<LessonModel>>(  
  stream: firestoreService.watchLessonsForStudent(student.academicYearId),  
  builder: (context, snapshot) {  
    final lessons = snapshot.data ?? [];  
    if (lessons.isEmpty) return const EmptyState(message: 'لا توجد دروس');  
    return ListView.separated(itemCount: lessons.length, /* ... */);  
  },  
)
```

 **Hero into the detail view:** When a lesson card is tapped and opens the PDF viewer, wrap the lesson icon/thumbnail on the list card and the equivalent element on the viewer screen in matching Hero tags built from the lesson id, e.g. Hero(tag: 'lesson-\${lesson.id}'). This reads as the card "opening into" the detail screen.

 **Try it yourself:** Build Lessons fully first. Then build Books, Videos, and Updates by copying Lessons' structure and swapping the model/query — notice how little actually changes.

STEP 9

The Events Feed

Goal: A feed screen with pinned items — introduces sorting client-side and pull-to-refresh.

Fixed events first

The query returns events for 'all' or the student's year, newest first. Before rendering, split the list into fixed vs. regular and concatenate fixed-first — Firestore doesn't need to know about "pinned", that's purely a client-side ordering step.

```
final fixed = events.where((e) => e.isFixed).toList();
final regular = events.where((e) => !e.isFixed).toList();
final ordered = [...fixed, ...regular];
```



RefreshIndicator: Wrap the ListView in a RefreshIndicator even though StreamBuilder already updates live — users on a spotty connection still expect the pull-to-refresh gesture to be there, and Flutter animates the spinner for you automatically.



Try it yourself: Give the "مثبت" (pinned) pill a subtle continuous animation — e.g. a slow opacity pulse using TweenAnimationBuilder — so pinned events draw the eye without being distracting.

STEP 10

The Notifications Screen

Goal: Merge two notification types into one feed, and add read/unread state.

One query, two origins

Automatic reminders and manual notifications both live in the same notifications collection, distinguished only by a type field. The student-facing screen doesn't care which produced a given notification — one StreamBuilder, one query on targetAudience, rendered with an icon that varies by type.

```
Icon(  
  n.type == 'automatic' ? Icons.alarm_rounded : Icons.campaign_rounded,  
  color: n.type == 'automatic' ? AppColors.blue : AppColors.goldDark,  
)
```

 **Pulsing unread dot:** A small unread-indicator dot benefits from a subtle looping scale/opacity animation (an outer glow that expands and fades) — build it once as a small StatefulWidget with an AnimationController set to repeat(reverse: true), and reuse it anywhere you need an "unread" or "live" indicator.

```
_controller = AnimationController(vsync: this, duration: const Duration(seconds:  
2))..repeat();
```

 **Try it yourself:** Tapping a notification should call `notificationService.markRead(id, uid)` — confirm the dot disappears live, without a manual refresh, because you're watching read-state as its own stream.

STEP 11

The Mentor Dashboard & CRUD Screens

Goal: Reuse everything you've learned so far to build the content-management side.

Same list pattern, plus a create/edit form

Every mentor content screen (Lessons, Books, Videos) is the Step 8 list pattern PLUS a floating action button that opens a bottom sheet form, PLUS a publish/unpublish Switch and a delete button on each row.

```
FloatingActionButton(  
  onPressed: () => showModalBottomSheet(  
    context: context,  
    isScrollControlled: true,  
    builder: (_) => LessonEditor(academicYearId: selectedYear),  
  ),  
  child: const Icon(Icons.add_rounded),  
)
```

 **Bottom sheets are already animated:** `showModalBottomSheet` slides its content up with a built-in animation — no extra work needed. What you **do** control is `isScrollControlled: true`, which lets the sheet grow to fit a form with several fields instead of being capped at half-screen.

The year-switcher tab

Since the one mentor manages all 3 years, every content screen needs a way to switch which year you're viewing. Build this once as a shared `YearTabBar` widget — three equal segments, the active one on a white pill with a soft shadow — and reuse it on Lessons, Books, and Videos screens.

 **Try it yourself:** Animate the active segment of `YearTabBar` sliding between positions with `AnimatedContainer` instead of instantly repainting — compare how much more polished it feels with a 200ms curve.

STEP 12

The Admin Screens

Goal: The smallest step — almost everything here reuses patterns from Steps 7–11.

- Admin Dashboard: same stat-card grid pattern as the Mentor Dashboard, different counts.
- Users screen: same list + Switch pattern as Mentor → Students, but across all roles instead of one.
- Notifications: identical to Mentor → Notifications, with one extra dropdown option (all_users) that mentors don't get.
- Academic Years: literally the same screen the mentor uses — don't rebuild it, import and reuse the widget.

 **Recognize the reuse:** If you find yourself about to copy-paste a whole screen file for Admin that's functionally identical to a Mentor one, that's a signal to extract a shared widget instead — see AdminAcademicYearsScreen just wrapping AcademicYearsScreen as an example.

 **Try it yourself:** Before writing any Admin screen, ask: "is this actually different from the Mentor version, or just relabeled?" Only write new code where the answer is genuinely yes.

STEP 13

Cloud Functions: Create Student & Send Notification

Goal: Move two operations server-side because the client should never be trusted with them.

Why these two need a server

- Creating a student account needs to create a Firebase Auth user — that requires the Admin SDK, which only runs in a trusted server environment, never in the Flutter app.
- Sending a manual notification needs to resolve "all students in year_2" into a list of FCM tokens and call the Messaging API — exposing that logic to the client would mean any signed-in user could theoretically spam notifications.

```
export const createStudentAccount = onCall(async (request) => {
  const callerRole = await getRole(request.auth?.uid);
  if (callerRole !== 'mentor' && callerRole !== 'admin') {
    throw new HttpsError('permission-denied', 'Not allowed');
  }
  const user = await admin.auth().createUser({ email, password });
  // then write users/{uid} and students/{uid} docs ...
});
```



Calling it from Flutter: The Flutter side just calls `FirebaseFunctions.instance.httpsCallable('createStudentAccount').call({...})` — all the sensitive logic and the permission check live only in the function, never in the app.

 **Try it yourself:** Deploy just this one function first (`firebase deploy --only functions:createStudentAccount`) and test it from the Firebase Console's "test function" panel before wiring up the Flutter button.

STEP 14

Automatic Learning Reminders

Goal: A scheduled Cloud Function — your first code that runs with no user present at all.

The mental model

This function doesn't respond to a button tap. Cloud Scheduler wakes it up on a cron schedule (e.g. every hour) regardless of whether anyone has the app open. Each run, it reads every `academicYears` document's configured `learningDay1/2` and `lessonStartTime`, and decides whether "right now" matches a reminder moment.

```
export const sendLearningReminders = onSchedule(  
  { schedule: 'every 60 minutes', timeZone: 'Asia/Beirut' },  
  async () => {  
    const years = await db.collection('academicYears').get();  
    for (const year of years.docs) {  
      // compare today/tomorrow's weekday against learningDay1/2  
      // → send to students where academicYearId == year.id  
    }  
  },  
);
```

 **Keep it separate on purpose:** Notice this writes notifications with `type: 'automatic'` and `senderId: 'system'` — the exact same collection the manual-send function writes to, but never through the same code path. That separation is what the spec means by "keep automatic reminders separate from manual notifications": one system, two independent triggers.

 **Try it yourself:** Temporarily change the schedule to ``every 2 minutes`` while testing locally with the Firebase emulator, so you don't have to wait an hour to see it fire.

STEP 15

Firestore & Storage Security Rules

Goal: Enforce on the server everything your app UI has been assuming.

The rule you'll write over and over

For every "students only see their own year's published content" query in your app, write the matching rule: read is allowed only if the requester's own students/{uid}.academicYearId equals the document's academicYearId, and isPublished == true.

```
match /lessons/{id} {
  allow read: if isMentorOrAdmin() ||
    (isStudent() && resource.data.isPublished == true
      && resource.data.academicYearId == myAcademicYear());
  allow write: if isMentorOrAdmin();
}
```

 **Test before you trust:** Use the Firestore Rules Playground in the console (or the emulator's rules unit tests) to try a request as a year_1 student against a year_2 lesson and confirm it's denied — don't rely on the app UI simply "not offering" the option, since that's not real security.

 **Try it yourself:** Deliberately try to break your own rules from the Flutter app — temporarily hard-code a query for another year's content and confirm Firestore itself rejects it, not just your UI hiding the button.

STEP 16

Test & Deploy

Goal: Bring every piece together and ship a build.

A practical test pass

- Seed the 3 academicYears documents and one mentor + one admin account manually.
- As the mentor: create one student in each year, add one lesson/book/video/update per year, send one manual notification.
- As each student: confirm you only ever see your own year's content — never another year's, published or not.
- Wait for (or fast-forward, via the emulator) an automatic reminder and confirm it lands only for the intended year.

```
flutter build apk --release  
# or  
flutter build ipa --release
```

 **Ship incrementally:** You don't need every step in this guide finished before your first real test with a real mentor account — Steps 0–7 alone already give you a login-to-dashboard flow worth testing on a real device early.

From here, the natural next additions are the two things this guide deliberately left as stubs: a real file-upload flow (`image_picker` + `StorageService`) instead of pasting URLs, and downloading a lesson PDF locally before handing it to `flutter_pdfview`. Both slot into the patterns you've already built — they're not a new architecture, just one more service call inside a screen you already know how to write.